МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность ПО-13

Выполнил:
М.А. Романюк
студент группы ПО-13
Проверил:
А.А. Крощенко
доц. кафедры ИИТ,
15.04.2026

Брест 2026

Цель работы: научиться работать с Github API, приобрести практические навыки написания программ для работы с REST API или GraphQL API

Общее задание: используя Github API, реализовать предложенное задание на языке Python. Выполнить визуализацию результатов, с использованием графика или отчета. Можно использовать как REST API (рекомендуется), так и GraphQL

Задание:

Вариант 9.

9. Автоматизированный мониторинг активности в GitHub-репозитории

Условие:

Напишите Python-скрипт, который анализирует активность в указанном GitHub-репозитории и отслеживает:

1. Новые коммиты
2. Созданные и закрытые issues
3. Открытые и закрытые pull requests
4. Новых контрибьюторов
5. Упоминания в комментариях (@username)
6. Количество звёзд и форков

Скрипт должен уметь:

- Подключаться к **GitHub API** и получать последние изменения в репозитории.
- Фильтровать события по времени (например, за последние 24 часа).

```
Введите репозиторий для мониторинга (owner/repo): django/django
Введите временной диапазон (в часах): 24
Мониторинг активности в "django/django" за последние 24 часа:
15 новых коммитов
5 новых pull requests (2 закрыто)
8 новых issues (3 закрыто)
2 новых контрибьютора: @newdev1, @newdev2
10 новых звёзд, 2 новых форка
3 упоминания пользователей: @maintainer, @contributor
```

Код программы

```
#!/usr/bin/env python3
"""
GitHub Repository Activity Monitor
"""

import argparse
import sys
from collections import Counter, defaultdict
from datetime import datetime, timedelta, timezone
from typing import Dict, List, Set, Tuple

import matplotlib.pyplot as plt
import requests

plt.rcParams['font.sans-serif'] = ['Arial Unicode MS', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False


class GitHubMonitor:
    """Class for monitoring GitHub repository activity"""

    def __init__(self, repo: str, token: str = None):
        """Initialize the monitor"""
        self.repo = repo
        self.base_url = "https://api.github.com"
        self.headers = {"Accept": "application/vnd.github.v3+json"}
        if token:
            self.headers["Authorization"] = f"token {token}"

        self.stats = {
            'commits': [],
            'pulls': {'opened': [], 'closed': []},
            'issues': {'opened': [], 'closed': []},
            'contributors': set(),
            'mentions': defaultdict(list),
            'stars': 0,
            'forks': 0
        }

    def _make_request(self, url: str, params: dict = None) -> dict:
        """Execute request to GitHub API"""
        try:
            response = requests.get(url, headers=self.headers, params=params, timeout=30)
            if response.status_code == 403 and 'rate limit' in response.text.lower():
                reset_time = int(response.headers.get('X-RateLimit-Reset', 0))
                if reset_time:
                    wait_time = reset_time - datetime.now().timestamp()
```

```
                if wait_time > 0:
                    print(f"\n[WARN] Limit exceeded! Wait {wait_time/60:.0f} min...")
                    return None
        response.raise_for_status()
        return response.json()
    except requests.exceptions.RequestException as e:
        print(f"Error: {e}")
        return None

def get_repo_info(self) -> Tuple[int, int]:
    """Get current stars and forks count"""
    url = f"{self.base_url}/repos/{self.repo}"
    data = self._make_request(url)
    if data:
        return data.get('stargazers_count', 0), data.get('forks_count', 0)
    return 0, 0

def get_commits(self, since: datetime) -> List[dict]:
    """Get commits for specified period"""
    url = f"{self.base_url}/repos/{self.repo}/commits"
    params = {'since': since.isoformat(), 'per_page': 100}
    commits = []
    page = 1

    while True:
        params['page'] = page
        data = self._make_request(url, params)
        if not data:
            break
        commits.extend(data)
        if len(data) < 100:
            break
        page += 1
    return commits

def get_pull_requests(self, since: datetime) -> Tuple[List, List]:
    """Get opened and closed PRs for period"""
    opened_prs = []
    closed_prs = []
    url = f"{self.base_url}/repos/{self.repo}/pulls"
    params = {'state': 'all', 'sort': 'created', 'direction': 'desc', 'per_page': 100}
    page = 1

    while True:
        params['page'] = page
        data = self._make_request(url, params)
        if not data:
            break
```

```
            for pr in data:
                created_at = datetime.fromisoformat(pr['created_at'].replace('Z',
'+00:00'))
                if created_at < since and page > 1:
                    return opened_prs, closed_prs

                if created_at >= since:
                    opened_prs.append(pr)

                if pr.get('closed_at'):
                    closed_at = datetime.fromisoformat(pr['closed_at'].replace('Z',
'+00:00'))
                    if closed_at >= since:
                        closed_prs.append(pr)

            if len(data) < 100:
                break
            page += 1
        return opened_prs, closed_prs

    def get_issues(self, since: datetime) -> Tuple[List, List]:
        """Get opened and closed issues for period"""
        opened_issues = []
        closed_issues = []
        url = f"{self.base_url}/repos/{self.repo}/issues"
        params = {'state': 'all', 'sort': 'created', 'direction': 'desc', 'per_page': 50}
        page = 1
        max_pages = 20

        while page <= max_pages:
            params['page'] = page
            data = self._make_request(url, params)
            if not data:
                break

            for issue in data:
                if 'pull_request' in issue:
                    continue

                created_at = datetime.fromisoformat(issue['created_at'].replace('Z',
'+00:00'))
                if created_at < since:
                    return opened_issues, closed_issues

                if created_at >= since:
                    opened_issues.append(issue)
```

```
                if issue.get('closed_at'):
                    closed_at = datetime.fromisoformat(issue['closed_at'].replace('Z',
'+00:00'))
                    if closed_at >= since:
                        closed_issues.append(issue)

            if len(data) < 50:
                break
            page += 1
        return opened_issues, closed_issues

    def extract_mentions(self, items: List[dict]) -> Dict[str, List[str]]:
        """Extract mentions from comments and descriptions"""
        mentions = defaultdict(list)
        for item in items:
            if 'body' in item and item['body']:
                self._find_mentions_in_text(item['body'], item.get('html_url', ''),
mentions)
            if 'comments_url' in item:
                comments = self._make_request(item['comments_url'])
                if comments:
                    for comment in comments:
                        if 'body' in comment and comment['body']:
                            self._find_mentions_in_text(
                                comment['body'], comment.get('html_url', ''), mentions)
        return mentions

    def _find_mentions_in_text(self, text: str, url: str, mentions: Dict) -> None:
        """Find mentions in text"""
        words = text.split()
        for word in words:
            if word.startswith('@') and len(word) > 1:
                username = word[1:].split('(')[0].split(',')[0].split(']')[0].strip()
                if username and not username.isdigit():
                    mentions[username].append(url)

    def get_contributors(self, commits: List[dict]) -> Set[str]:
        """Extract contributors from commits"""
        contributors = set()
        for commit in commits:
            if commit.get('author') and commit['author'].get('login'):
                contributors.add(commit['author']['login'])
            elif commit.get('commit', {}).get('author', {}).get('name'):
                contributors.add(commit['commit']['author']['name'])
        return contributors

    def monitor_activity(self, hours: int = 24) -> dict:
        """Main monitoring method"""
```

```
        since_time = datetime.now(timezone.utc) - timedelta(hours=hours)

        print(f"\nMonitoring: {self.repo}")
        print(f"Period: {hours} hours\n")

        current_stars, current_forks = self.get_repo_info()
        commits = self.get_commits(since_time)
        opened_prs, closed_prs = self.get_pull_requests(since_time)
        opened_issues, closed_issues = self.get_issues(since_time)
        contributors = self.get_contributors(commits)

        all_items = opened_prs + closed_prs + opened_issues + closed_issues
        mentions = self.extract_mentions(all_items)

        self.stats['commits'] = commits
        self.stats['pulls']['opened'] = opened_prs
        self.stats['pulls']['closed'] = closed_prs
        self.stats['issues']['opened'] = opened_issues
        self.stats['issues']['closed'] = closed_issues
        self.stats['contributors'] = contributors
        self.stats['mentions'] = mentions
        self.stats['stars'] = current_stars
        self.stats['forks'] = current_forks

        return self.stats

    def print_report(self) -> None:
        """Print report"""
        print("\n" + "=" * 80)
        print(f"REPORT: {self.repo}")
        print("=" * 80)

        print(f"\nCOMMITS: {len(self.stats['commits'])}")
        if self.stats['commits']:
            for commit in self.stats['commits'][:3]:
                msg = commit['commit']['message'].split('\n')[0][:50]
                print(f"   * {msg}")

        print("\nPULL REQUESTS:")
        print(f"   Opened: {len(self.stats['pulls']['opened'])}")
        print(f"   Closed: {len(self.stats['pulls']['closed'])}")

        print("\nISSUES:")
        print(f"   Opened: {len(self.stats['issues']['opened'])}")
        print(f"   Closed: {len(self.stats['issues']['closed'])}")

        print(f"\nCONTRIBUTORS: {len(self.stats['contributors'])}")
        if self.stats['contributors']:
```

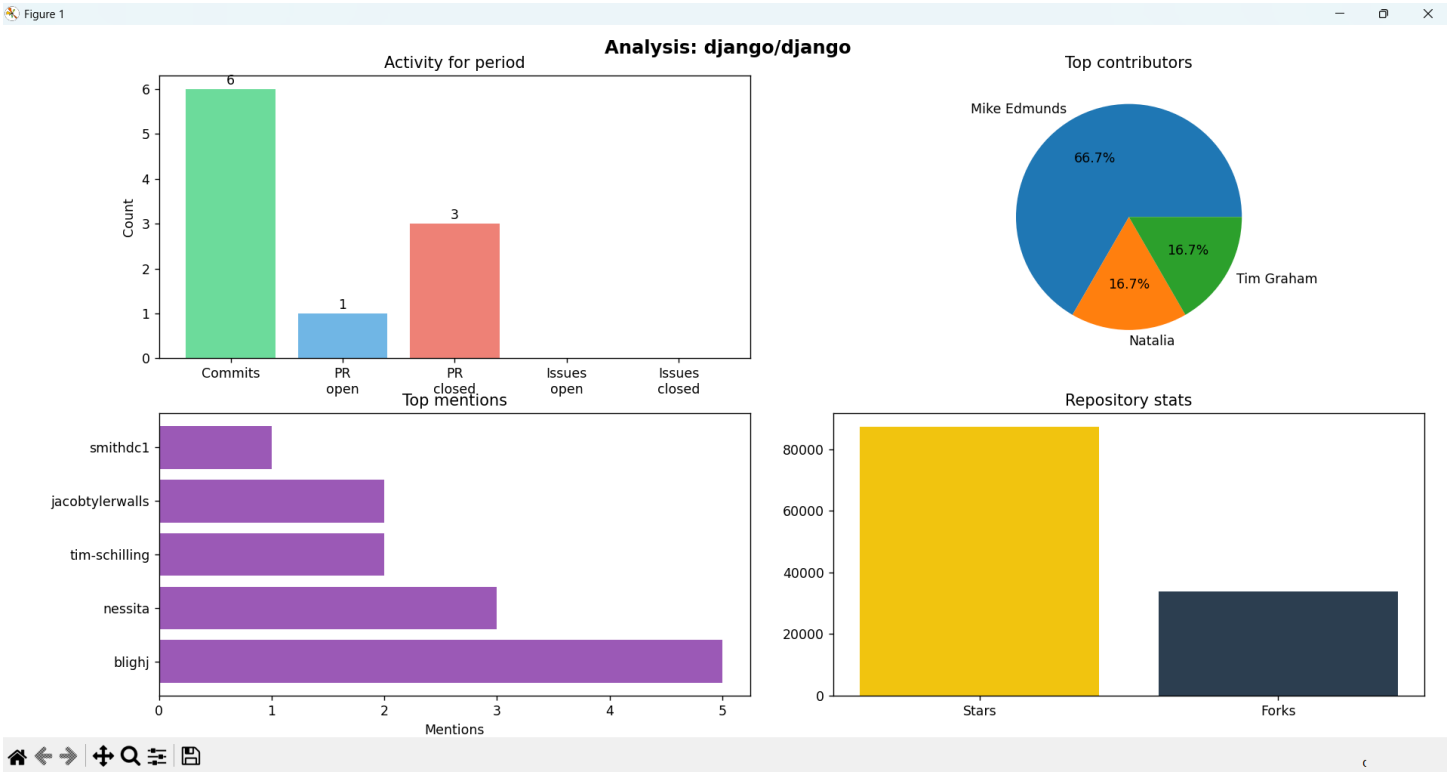
```
            contrib_list = ', '.join([f'@{c}' for c in
list(self.stats['contributors'])[:5]])
            print(f"   {contrib_list}")

        total_mentions = sum(len(urls) for urls in self.stats['mentions'].values())
        print(f"\nMENTIONS: {total_mentions}")

        print("\nREPOSITORY STATS:")
        print(f"   Stars: {self.stats['stars']}")
        print(f"   Forks: {self.stats['forks']}")
        print("\n" + "=" * 80)

    def visualize(self) -> None:
        """Visualize data"""
        fig, axes = plt.subplots(2, 2, figsize=(12, 10))
        fig.suptitle(f'Analysis: {self.repo}', fontsize=14, fontweight='bold')

        # Activity graph
        ax1 = axes[0, 0]
        categories = ['Commits', 'PR\nopen', 'PR\nclosed', 'Issues\nopen',
'Issues\nclosed']
        values = [
            len(self.stats['commits']),
            len(self.stats['pulls']['opened']),
            len(self.stats['pulls']['closed']),
            len(self.stats['issues']['opened']),
            len(self.stats['issues']['closed'])
        ]
        colors = ['#2ecc71', '#3498db', '#e74c3c', '#3498db', '#e74c3c']
        plot_bars = ax1.bar(categories, values, color=colors, alpha=0.7)
        ax1.set_ylabel('Count')
        ax1.set_title('Activity for period')
        for bar_obj, val in zip(plot_bars, values):
            if val > 0:
                ax1.text(bar_obj.get_x() + bar_obj.get_width()/2,
                        bar_obj.get_height() + 0.1, str(val), ha='center')

        # Top contributors
        ax2 = axes[0, 1]
        if self.stats['commits']:
            counts = Counter()
            for commit in self.stats['commits']:
                counts[commit['commit']['author']['name']] += 1
            top = dict(counts.most_common(5))
            if top:
                ax2.pie(top.values(), labels=top.keys(), autopct='%1.1f%%')
                ax2.set_title('Top contributors')
```

```
        # Mentions
        ax3 = axes[1, 0]
        if self.stats['mentions']:
            top = dict(sorted(self.stats['mentions'].items(),
                            key=lambda x: len(x[1]), reverse=True)[:5])
            ax3.barh(list(top.keys()), [len(v) for v in top.values()], color='#9b59b6')
            ax3.set_xlabel('Mentions')
            ax3.set_title('Top mentions')

        # Repository stats
        ax4 = axes[1, 1]
        ax4.bar(['Stars', 'Forks'], [self.stats['stars'], self.stats['forks']],
                color=['#f1c40f', '#2c3e50'])
        ax4.set_title('Repository stats')

        plt.tight_layout()
        plt.show()


def main() -> None:
    """Main function"""
    parser = argparse.ArgumentParser(description='GitHub Monitor')
    parser.add_argument('repo', nargs='?', help='owner/repo')
    parser.add_argument('-t', '--token', help='GitHub token')
    parser.add_argument('-H', '--hours', type=int, default=24, help='Hours (default: 24)')
    args = parser.parse_args()

    repo = args.repo
    if not repo:
        repo = input("Repository (owner/repo): ").strip()

    hours = args.hours
    if not args.repo:
        h = input(f"Hours ({hours}): ").strip()
        if h:
            hours = int(h)

    monitor = GitHubMonitor(repo, args.token)

    try:
        monitor.monitor_activity(hours)
        monitor.print_report()

        show = input("\nShow graphs? (y/n): ").strip().lower()
        if show == 'y':
            monitor.visualize()
    except KeyboardInterrupt:
        print("\n\nInterrupted")
```

```
        sys.exit(0)
    except Exception as e:
        print(f"\nError: {e}")
        sys.exit(1)


if __name__ == "__main__":
    main()
```

Рисунки с результатами работы программы

```
Monitoring: django/django
Period: 12 hours


============================================================
REPORT: django/django
============================================================

COMMITS: 6
   * Refs #35514 -- Cleaned up email docs.
   * Refs #35514 -- Added "Configuring email" section t
   * Refs #35514 -- Moved and renamed "Sending multiple

PULL REQUESTS:
   Opened: 1
   Closed: 3

ISSUES:
   Opened: 0
   Closed: 0

CONTRIBUTORS: 3
   @medmunds, @timgraham, @nessita

MENTIONS: 14

REPOSITORY STATS:
   Stars: 87281
   Forks: 33817

============================================================
```



Вывод: научился работать с Github API, приобрел практические навыки написания программ для работы с REST API или GraphQL API